



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Collections

Collections Algorithm

Pemrograman Berorientasi Objek

Algorithm

- Sorting, mengurutkan elemen
- Shuffling, kebalikan dari sorting (mengacak urutan)
- Routine Data Manipulation: reverse, fill, copy, swap, addAll
- Searching, mencari elemen
- Composition: frequency, disjoint
- Finding Extreme Value: min, max

Contoh

```
import java.util.*;

public class SetTest {
    public static void main(String[] args) {
        String[] list = { "John", "Penelope", "April", "Zachary", "George" };
        System.out.println("entry order: " + Arrays.toString(list) + "\n");

        Set<String> people;

        people = new HashSet<String>(); // put into a HashSet
        for (String elt: list) people.add(elt);
        System.out.println("HashSet order: " + people);

        // print the set by iteration
        for (String person: people)
            System.out.println("\t" + person);
        System.out.println();

        people = new LinkedHashSet<String>(); // put into a LinkedHashSet
        for (String elt: list) people.add(elt);
        System.out.println("LinkedHashSet order: " + people + "\n");

        people = new TreeSet<String>(); // put into a TreeSet
        for (String elt: list) people.add(elt);
        System.out.println("TreeSet order: " + people);
    }
}
```

```
import java.util.*;
```

```
public class MapTest {
    static class Pair<T1,T2> {    // declared static to work within main
        T1 first; T2 second;
        Pair(T1 first, T2 second) { this.first = first; this.second = second; }
        public String toString() { return first + "=" + second; }
    }

    public static void main(String[] args) {
        List<Pair<String,Integer>> list = new ArrayList<Pair<String,Integer>>();
        list.add(new Pair<String,Integer>("John",33));
        list.add(new Pair<String,Integer>("Stephen",71));
        list.add(new Pair<String,Integer>("Melissa",15));
        list.add(new Pair<String,Integer>("George",40));
        list.add(new Pair<String,Integer>("Alison",27));

        System.out.println("entry order: " + list + "\n");

        Map<String,Integer> age;

        age = new LinkedHashMap<String,Integer>();    // as a LinkedHashMap
        for (Pair<String,Integer> p: list) age.put(p.first, p.second);

        System.out.println("LinkedHashMap order: " + age.entrySet());
        System.out.println("keys order: " + age.keySet() + "\n");
    }
}
```

Contoh

Contoh

```
System.out.println("iteration type 1");    // iterate on the entrySet
for (Map.Entry<String,Integer> entry: age.entrySet())
    System.out.println("\tkey=" + entry.getKey() + "\tvalue=" + entry.getValue());
System.out.println();
```

```
System.out.println("iteration type 2");    // iterate on the keySet
for (String key: age.keySet())
    System.out.println("\tkey=" + key + "\tvalue=" + age.get(key));
System.out.println();
```

```
age = new HashMap<String,Integer>();    // as a HashMap
for (Pair<String,Integer> p: list) age.put(p.first, p.second);
System.out.println("HashMap order: " + age.entrySet());
```

```
age = new TreeMap<String,Integer>();    // as a TreeMap
for (Pair<String,Integer> p: list) age.put(p.first, p.second);
System.out.println("TreeMap order: " + age.entrySet());
```

```
}
```

```
}
```



Contoh

```
import java.util.*;

public class SortTest {

    public static void main(String[] args) {
        // create list of random integers and sort them
        Random r = new Random();

        List<Integer> list = new ArrayList<Integer>();

        for (int i=0; i<10; i++)
            list.add(new Integer(r.nextInt() % 100));

        System.out.print("the list: "); System.out.println(list);

        Collections.sort(list);
        System.out.print("sorted: "); System.out.println(list);

    }
}
```

Terima Kasih



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Collections

Collections

Pemrograman Berorientasi Objek

Collections

- Disebut juga container
- Objek yang mengelompokkan sekumpulan elemen menjadi sebuah unit
- Digunakan untuk menyimpan, mengambil, memanipulasi, dan mengkomunikasikan agregasi data
- Biasanya merepresentasikan data dalam bentuk grup



Collections Framework

- Arsitektur *unified* untuk merepresentasikan dan memanipulasi collections
- Terdiri atas: interfaces, implementations, algorithms

Collections Framework

- Interfaces: ADT yang merepresentasikan collections
- Implementations: implementasi konkrit dari collection interface (struktur data yang reusable)
- Algorithms: method yang melakukan komputasi atas objek yang mengimplementasikan collection interface

Keuntungan Collections Framework

- Mereduksi usaha memrogram
- Meningkatkan kecepatan dan kualitas program
- Interoperability antar API yang tak terkait
- Mereduksi usaha memahami dan menggunakan API baru
- Mereduksi usaha mendesain API baru
- Meningkatkan software reuse

Terima Kasih



Riza Satria Perdana, S.T., M.T.

Teknik Informatika - STEI ITB

Collections

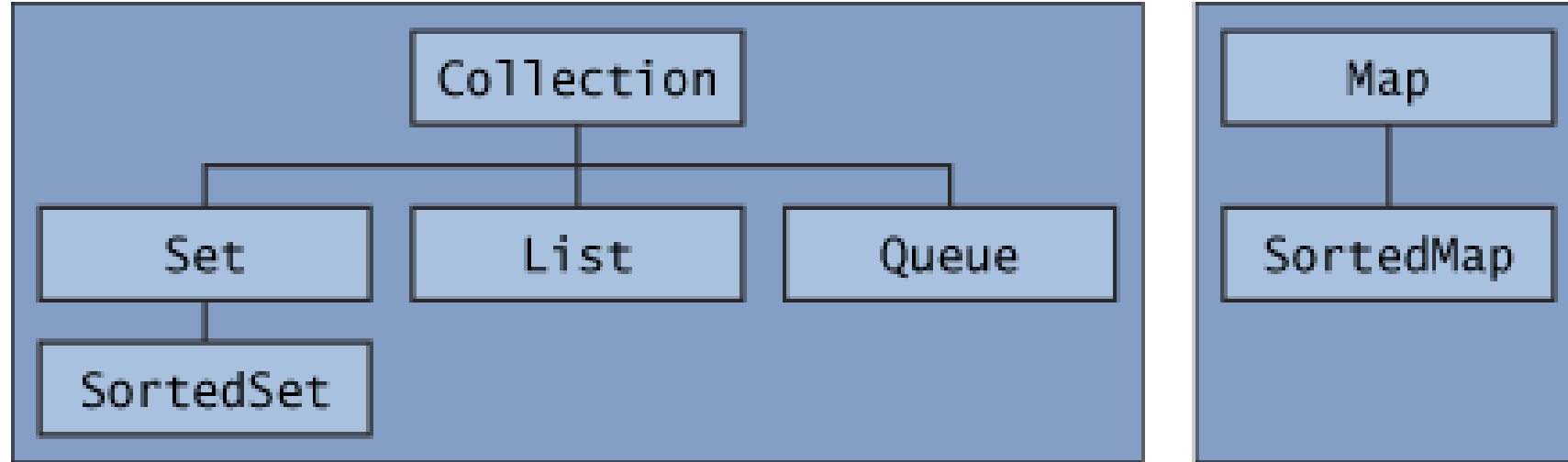
Collections Interface & Implementation

Pemrograman Berorientasi Objek



Interfaces

- Core collection interface mengenkapsulasi tipe berbeda collections



The core collection interfaces.

Interfaces

- Collection, root dari collections hirarki merepresentasikan group of object (elemen)
- Set, collections yang tidak boleh ada duplikasi elemen
- List (sequence), collections yang elemennya teratur, boleh duplikasi

Interfaces

- Queue, mengurutkan elemen berdasarkan FIFO
- Map (hashtable), memetakan key ke value, tidak boleh duplikasi
- SortedSet, Set yang terurut
- SortedMap, Map yang terurut

Collection Interface

```
public interface Collection<E> extends Iterable<E> {  
    // basic operations  
    int size();  
    boolean isEmpty();  
    boolean contains(Object element);  
    boolean add(E element);  
    boolean remove(Object element);  
    Iterator<E> iterator();  
  
    // bulk operations  
    boolean containsAll(Collection<?> c);  
    boolean addAll(Collection<? extends E> c);  
    boolean removeAll(Collection<?> c);  
    boolean retainAll(Collection<?> c);  
    void clear();  
  
    // array operations  
    Object[] toArray();  
    <T> T[] toArray(T[] a);  
}
```

Traversing Collections

- Konstruksi for-each

```
for (Object o : collection)
    System.out.println(o);
```

- Iterator

```
public interface Iterator<E> {
    boolean hasNext();
    E next();
    void remove(); //optional
}

static void filter(Collection<?> c) {
    for (Iterator<?> it = c.iterator(); it.hasNext(); )
        if (!cond(it.next()))
            it.remove();
}
```

Implementations

- Data objek yang digunakan untuk menyimpan collections
- Mengimplementasikan interface yang dibahas sebelumnya
- Ada beberapa macam implementation: general purpose, special purpose, concurrent, wrapper, convenience, abstract

General Purpose

General-purpose Implementations

Interfaces	Implementations				
	Hash table	Resizable array	Tree	Linked list	Hash table + Linked list
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList		LinkedList	
Queue					
Map	HashMap		TreeMap		LinkedHashMap



Terima Kasih